

Divide and Conquer

Kanishk Goel

22 February, 2026

1 MergeSort

Problem: Sorting

Input: An array of n numbers in arbitrary order.

Output: Array with the same numbers, sorted in ascending order

MergeSort

Input: Array A of n distinct integers.

Output: Array with the same integers, sorted in ascending order
--

if $n \leq 1$ then

return A

$C :=$ recursively sort first half of A

$D :=$ recursively sort second half of A
--

return Merge(C, D)

Merge

Input: Sorted arrays C and D (length $n/2$ each)

Output: Sorted array B (length n)

Simplifying assumption: n is even
--

$i := 1$

$j := 1$

for $k := 1$ to n do

if $i \leq n/2$ and $(j > n/2$ or $C[i] \leq D[j])$ then
--

$B[k] := C[i]$

$i := i + 1$

else

$B[k] := D[j]$

$j := j + 1$

Problem: Counting Inversions
Input: An array A of distinct integers. Output: The number of inversions of A i.e pairs (i, j) of array indices with $i \leq j$ and $A[i] \geq A[j]$.

Used in Collaborative Filtering which is a technique for generating recommendations.

CountInversions
Input: array A of n distinct integers. Output: sorted array B with the same integers, and the number of inversions of A . if $n = 0 \vee n = 1$ then return $(A, 0)$ else $(C, leftInv) := \text{CountInversions}(\text{first half of } A)$ $(D, rightInv) := \text{CountInversions}(\text{second half of } A)$ $(B, splitInv) := \text{CountSplitInversions}(C, D)$ return $(B, leftInv + rightInv + splitInv)$

CountSplitInversions
Input: sorted arrays C and D (length $n/2$ each). Output: sorted array B (length n). Simplifying assumption: n is even $i := 1, j := 1, splitInv := 0$ for $k := 1$ to n do if $C[i] < D[j]$ then $B[k] := C[i], i := i + 1$ else $B[k] := D[j], j := j + 1$ $splitInv := splitInv + (\frac{n}{2} - i + 1)$ return $(B, splitInv)$

The idea used is equivalent to merge subroutine of mergesort.

2 QuickSort

The pseudocode for quicksort:

QuickSort
Input: array A of n distinct integers, left and right endpoints $l, r \in \{1, 2, 3, \dots, n\}$. Output: Elements of the subarray $A[l], A[l + 1], \dots, A[r]$ are sorted from smallest to largest. if $l \geq r$ then return $i \leftarrow \text{ChoosePivot}(A, l, r)$ swap $A[l]$ and $A[i]$ $j \leftarrow \text{Partition}(A, l, r)$ QuickSort($A, l, j - 1$) QuickSort($A, j + 1, r$)

Partition
Input: array A of n distinct integers, left and right endpoints $l, r \in \{1, 2, \dots, n\}$ with $l \leq r$. Output: Elements of the subarray $A[l], \dots, A[r]$ are partitioned around $A[l]$. Final output is the position of the pivot element. $p \leftarrow A[l]$ $i \leftarrow l + 1$ for $j \leftarrow l + 1$ to r do if $A[j] \geq p$ then swap $A[j]$ and $A[i]$ $i \leftarrow i + 1$ swap $A[l]$ and $A[i - 1]$ return $i - 1$

Now the idea of choosing pivot is important, randomized pivot selection has an average running time of $O(n \log n)$

3 Selection Problem

Problem: Selection
Input: An array A of n numbers, in arbitrary order, and an integer $i \in \{1, 2, \dots, n\}$. Output: The i^{th} smallest element of A or the i^{th} order statistic.

There are two algorithms one is RSelect which is a randomized select algorithm with an average case of $O(n)$. The other is DSelect.

RSelect
Input: array A of $n \geq 1$ distinct numbers, and an integer $i \in \{1, 2, \dots, n\}$ Output: the i^{th} order statistic of A. <pre> if $n = 1$ then return $A[1]$ choose pivot element p at random from A partition A around p $j \leftarrow p$'s position in partitioned array if $j = i$ then return p else if $j > i$ then return $RSelect(\text{first part of } A, i)$ else return $RSelect(\text{second part of } A, i)$ </pre>

DSelect
Input: array A of $n \geq 1$ distinct numbers, and an integer $i \in \{1, 2, \dots, n\}$ Output: the i^{th} order statistic of A. <pre> if $n = 1$ then return $A[1]$ for $h \leftarrow 1$ to $\frac{n}{5}$ do $C[h] \leftarrow$ middle element from the h^{th} group of 5 $p \leftarrow DSelect(c, \frac{n}{10})$ partition A around p $j \leftarrow p$'s position in partitioned array if $j = i$ then return p else if $j > i$ then return $DSelect(\text{first part of } A, i)$ else return $DSelect(\text{second part of } A, i)$ </pre>